

READER EDITION

Data analytics agent review packet

Two detailed architecture reviews, presented for focused reading on desktop or phone. The original Markdown is embedded in full within this offline file.

● 2 reports ● 5.6k words ● Offline-ready ● Mobile-friendly

DETAILED REVIEW

Architecture & product review

The baseline assessment of the current agent's architecture and product direction.

Data analytics agent: architecture and product review

Reviewed September 10, 2026. This is an assessment and proposed roadmap; application behavior has not been changed.

The application has a strong foundation for a local analyst: durable evidence, explicit specialist responsibilities, iterative analysis, recoverable execution, and shared chart/report artifacts. Its largest remaining gaps are analytical verification and the experience of using the results. More autonomous execution will be valuable after those gaps are addressed.

“Top notch” should mean that users can consistently get the right business answer, understand its scope and limitations, inspect its evidence, and use it to make their next decision. The current implementation has not yet demonstrated that level of reliability across realistic workloads. There is no head-to-head benchmark here that establishes a ranking against commercial products.

Review basis

- Inspected coordinator and specialist construction, prompts and skills, semantic discovery, SQL validation, Python execution, persistence, lifecycle, publication, chart/report contracts, UI components, and tests.
- Ran the deterministic suite: **188 passed, 6 skipped, 4 warnings**. The warnings concern experimental streaming and dependency deprecations. Ruff's configured F checks passed.
- Read the previous live evaluation findings. Those trials establish useful repair behavior on synthetic data, not general analytical reliability.
- Inspected the current Streamlit UI using a temporary copy of an existing synthetic forecast. The older fixture needed its saved turn ID aligned with the current contract in that temporary copy. No compatibility changes were made to the application.
- Inspected desktop onboarding, the saved answer, chart/report presentation, and evidence navigation. This was not a comprehensive mobile, keyboard, or screen-reader audit.
- Checked current primary documentation from Snowflake, Databricks, Hex, LangChain, and Anthropic. Product documentation establishes available patterns, not independently verified superiority.
- No new live model trials or business warehouse queries were run. Local fixture inspection supplied the numerical examples below.

Installed stack: Deep Agents 0.7.0, LangChain 1.3.14, LangGraph 1.2.6, Streamlit 1.59.2, Plotly 6.9.0.

What is already strong

AREA	ASSESSMENT
Responsibility boundaries	Coordinator owns the answer; SQL owns source retrieval; Python operates on named saved inputs. This makes scope and provenance easier to reason about.
Evidence	Typed Parquet, explicit parent relationships, bounded previews, complete downloads, and separate chart presentation datasets are solid choices.
Statistical workflow	The specialist can inspect, execute, revise, request more source data, and continue. The skill covers leakage, baselines, temporal validation, and uncertainty.
Recovery	Checkpoints, committed tool outputs, Stop/Resume, durable reviews, correction delivery, and independent report retry address important real-world failures.
Presentation	Chat and reports reuse chart specifications; reports bind metric values to stored cells, escape text, check theme contrast, and preserve artifacts.
UI engineering	Stable widget identities, an independent composer, incremental event cursors, and lazy activity/evidence panels are thoughtful foundations.

Keep the current coordinator/SQL/Python topology. Additional permanent specialist agents would need evidence of better outcomes to justify their coordination overhead. Deep Agents remains an appropriate harness for this workload; its current documented capabilities include durable orchestration, context management, skills, and steering. [Deep Agents overview](#).

Highest-priority findings

1. Verify the meaning of claims, not just their artifact references

High priority; observed defect in a saved report and a current contract gap.

The inspected forecast report displays **340.00** under "Average 2025 forecast." Its narrative states an average of **351.000**, and the saved forecast column's mean is **351.0**. The report specification binds the card to row zero of the forecast column: January's value is 340.

The renderer faithfully displays that cell. The problem is that a valid cell reference does not establish that the label describes it correctly. Free-text answers, interpretations, labels, and percentage changes have similar exposure to unsupported claims. This is a historical generated example, not a claim that a new model run today would reproduce the same mistake. The current contract still permits it.

Evidence: [metric renderer](#), [metric schema](#), [answer contract](#).

Introduce a small typed claim record: metric identity, business label, value reference, unit, population/time scope, aggregation, and validation status. Bind report cards and material numerical statements to these claims. Have SQL or Python save the actual aggregate as evidence; the report renderer should not silently calculate business metrics. Generate standard labels from the metric definition, and validate narrative/claim agreement before publication. An LLM review can check interpretation, but arithmetic and reference checks should be deterministic.

Acceptance example: an "average" card cannot bind to an arbitrary monthly row; changing that binding causes validation to fail or explicitly changes the label. Report rendering retries must continue to reuse evidence.

2. Move core business semantics into execution checks

High priority; confirmed by code inspection and a local validator probe.

The source SQL validator enforces a single read-only query and rejects dangerous statement structures. It does not receive the semantic catalog. A query referencing an undeclared object, a potentially inflating orders-to-items SUM, and `SELECT *` all passed this structural validator. These probes were parsed locally, not executed against a source. Database privileges remain a separate boundary.

The semantic catalog contains useful definitions and relationships, but the application relies heavily on prompt adherence for source-object selection, join choice, business grain, and aggregation. This is the central correctness risk for ordinary analytics, including queries that execute successfully.

Evidence: [validator](#), [source execution tool](#), [semantic types](#).

Start with catalog-aware object/field checks and a small library of verified, parameterized business queries. For governed metrics, represent metric, dimensions, filters, and time grain explicitly. Validate supported join paths, uniqueness assumptions, aggregate grain, denominator definitions, and time boundaries. Use warehouse semantic execution where supported; do not attempt to build a universal semantic compiler before a focused version works end to end. Custom exploratory SQL still needs explicit scope and checks.

Snowflake's semantic routing prioritizes governed metric/join definitions; that capability is marked **Preview**, and its documentation explicitly describes limitations. Databricks provides trusted parameterized queries and functions. These are useful design references, not guarantees that generated answers are correct. [Snowflake semantic routing](#), [Databricks trusted assets](#).

Acceptance examples: revenue remains unchanged when adding a one-to-many dimension; weighted rates use the correct denominator; ambiguous fiscal periods trigger clarification; undeclared physical objects are rejected at execution.

3. Treat analytical quality evaluations as a release requirement

High priority; current coverage does not establish broad reliability.

The deterministic tests are valuable. The live suite is much narrower: five case categories over one small, regular synthetic fixture. Statistical cases largely check for terminology in saved outputs/code and the presence of executions. The descriptive case checks whether the expected number appears somewhere in the evidence, rather than fully verifying the final answer. These checks can pass despite misleading wording or invalid methodology.

Evidence: [live evaluations](#), [prior evaluation review](#).

Build a versioned set of real business question patterns, using authorized or synthetic datasets. Start with roughly 30–50 representative cases and several phrasings, then expand from failures. Separate development examples from held-out evaluation cases. Include:

- Joins, distinct counts, weighted metrics, nulls, missing categories, currencies, time zones, partial periods, and “top”/tie semantics.
- Follow-up scope, freshness requests, corrections, clarification, chart-only revision, and restart/recovery.

- Noisy forecasts, structural breaks, intermittent demand, short histories, leakage, multiple comparisons, and insufficient evidence.
- Agreement among the answer, report metrics, chart labels, and actual saved results.

Use result equivalence and invariants for SQL; persisted split definitions, predictions, and recomputed errors for models; calibrated human/model rubrics for explanation quality. Measure first-attempt and repeated-trial success, unnecessary clarification, repair rate, time to usable findings, total time, and cost per successful task. Keep adversarial cases outside tuning examples.

Databricks documents result-comparison benchmarks; Hex documents version-controlled evaluation suites and tracking consistency, runtime, and credit usage. Anthropic recommends assessing outcomes and using complementary graders across repeated trials. [Genie benchmarks](#), [Hex evals](#), [Agent evaluation guidance](#).

4. Make uncertainty a first-class chart concept

High priority for forecasting; confirmed current implementation limitation.

The renderer hard-codes the legend text **"Uncertainty interval"** for all bounds. The inspected artifact uses those bounds for a planning sensitivity range, explicitly described elsewhere as not a calibrated prediction interval. The chart vocabulary undermines that distinction.

The schema also restricts bands to a single-series line chart. The saved forecast reports that historical actuals were omitted to fit this restriction. Its historical dates remain on the x-axis, leaving a large span without plotted actual values.

Evidence: [band renderer](#), [band restriction](#).

Add explicit band kind, label, method, and optional coverage level. Support actuals, forecast, baseline, and bounds as a small set of typed chart layers, with a forecast boundary marker. Keep one authoritative chart specification for chat and report. Plotly already supplies the trace primitives; this needs a better application contract, not another charting library.

Also let the analysis specialist inspect a bounded diagnostic image when needed. Currently `AnalysisOutput.model_facing()` replaces its image path with `rendered: true`; the agent receives no image in that response. Numerical diagnostics remain essential, but the workflow should permit visual examination of residual and distribution plots. [Model-facing image output](#).

5. Turn the UI into an analytical workspace

High product priority; observed in the current UI.

The theme and native components are coherent. The main issue is hierarchy and interaction. Onboarding foregrounds framework details and execution modes; saved conversations sit below substantial technical information. A completed forecast contains a long answer, separate assumptions/interpretation, repeated analysis panels, a chart, an expanded report, and eight evidence sections. Dataset names look like internal variable names. The report preview occupies 900 pixels by default, and some report analysis sections include raw diagnostic output.

Evidence: [header/sidebar](#), [report preview](#), [answer assembly](#).

Use this hierarchy:

1. A concise answer and the most useful metric/chart.
2. A compact scope row: source, data as-of time, business period, filters, units, and validation state.
3. Relevant actions: change period, compare, break down, inspect data, refresh, export.
4. An adjacent or switchable workspace for **Findings / Data / Method / Report**.

Retain all evidence, but group intermediate artifacts under lineage instead of displaying every ancestor as a peer finding. Keep the HTML report mandatory under current requirements; make its default chat presence compact, with preview on demand. Use business labels and display formatting for dates, currency, precision, and percentages while keeping downloaded values exact.

Provide direct chart type/title/axis controls and selection-to-follow-up. A title-only edit should update the saved chart version and report without an LLM or warehouse round trip. Scope-changing actions should clearly state whether they filter a snapshot or fetch new data. Show clarification choices as actual choices, with free text still available.

Move saved conversations above diagnostics, add search and rename, and use a small status indicator with one technical disclosure. Preserve the stable composer, Activity disclosure, Stop/Resume, and report retry. For common tasks, prototype these changes in Streamlit before considering a frontend rewrite.

Hex documents manual chart exploration and side-by-side conversation/notebook/app views. The transferable pattern is direct manipulation combined with conversational analysis. [Hex Threads](#).

Architecture improvements to support those priorities

Persist analytical scope explicitly

Saved datasets already retain source, creation time, lineage, and executed code; runs also bind a catalog hash. Extend this into a compact analytical scope record: population, filters, observation grain, business date window, source freshness, units, metric identities, semantic version, and completeness.

Distinguish extraction time from data freshness. A dataset downloaded today may contain observations ending last year. Reuse decisions should be checked against this record, not inferred only from narrative and SQL.

Follow-up context currently replays prior answer text and artifact IDs but omits separate assumptions, interpretation, `partial`, and unresolved questions. Preserve these fields in the compact context so later answers do not accidentally drop caveats. For long conversations, select relevant prior evidence and maintain a bounded summary instead of rebuilding an ever-growing transcript. [Follow-up context](#).

Keep orchestration adaptive; make publication predictable

Preserve flexible SQL/Python investigation loops. Use deterministic validation at the execution and publication boundaries. A single mandatory critic agent would add cost and may share the original model's mistakes; invoke targeted review when the task, ambiguity, or validation results justify it.

Simple totals should take a short route through grounded SQL, typed findings, and a standard compact report. Preserve early findings and separate report retry. Remove repeated prompt sections and avoid asking the model to design a layout for every scalar answer.

All roles currently share one model object, and OpenAI reasoning effort is set to medium. After establishing the benchmark, compare a fast model for straightforward retrieval/presentation with a stronger configuration for difficult analysis. Adopt routing only if the measured accuracy/latency/cost tradeoff improves. No specific model is declared best by this review. [Model configuration](#).

Isolate Python before expanding unattended use

The fresh worker process, cleaned environment, time limit, cancellation, and output limits are valuable. They do not restrict the process's operating-system file access or network access. Generated code is executed with ordinary Python `exec`; memory and CPU consumption are not contained by the output-size checks.

This limitation is acknowledged in the project's local single-user scope. Before accepting untrusted uploads, broad connector content, or unattended jobs, put execution in an established isolation boundary with read-only input mounts, a bounded output directory, resource limits, controlled egress, and no service credentials. Preserve the current named-dataset contract. Keep warehouse credentials and access control outside generated Python. [Worker execution](#), [runner](#), [Deep Agents sandboxes](#).

For team deployment, add identity-scoped authorization for queries and artifact downloads, warehouse roles/masking, and durable job ownership before lifting the documented single-process constraint. Local SQLite is a reasonable fit for today's deployment; a distributed infrastructure rewrite is not the first improvement.

Build a reviewed knowledge improvement loop

Today, corrections influence the conversation but do not become a reusable, reviewed body of business knowledge. Add a workflow to turn accepted corrections into candidate metric definitions, synonyms, join guidance, and verified query examples. Give each entry scope, version, owner, and evaluation coverage. Validate it before it influences other conversations.

The existing lexical metadata search is a sensible baseline for the small catalogs. Add hybrid retrieval/reranking only when retrieval tests show that vocabulary or catalog size requires it. Keep verified examples distinct from held-out evaluation answers. Snowflake's verified query repository is an established reference for curating useful SQL examples. [Snowflake verified queries](#).

Proposed target workflow

MERMAID

```
flowchart TD
    U[Question or direct workspace action] --> C[Coordinator resolves scope and intent]
    C --> K[Curated definitions and verified examples]
    K --> S[SQL specialist: retrieve or reshape]
    S --> G[Scope and semantic checks]
    G --> D[Saved typed evidence and quality profile]
    D --> P[Python specialist when analysis requires it]
    P --> D
    D --> V[Validate claims, methodology and presentation semantics]
    V --> F[Publish findings]
    F --> R[Render report from shared evidence]
    F --> W[Interactive analytical workspace]
    R --> W
    W --> C
```

These boxes describe responsibilities. They do not require a new agent, service, or graph node for every box. Add small modules around existing execution, publication, and presentation contracts.

Sequence the work in complete increments

STAGE	DELIVERABLE	EXIT CRITERIA
1. Trust and baseline	Claim bindings, corrected uncertainty semantics, representative eval suite, preserved follow-up caveats	Catch the saved report's wrong-average binding; reject mislabeled bands; reproduce an evaluated baseline.
2. Business correctness	Catalog-aware source validation, a focused set of governed metrics and verified queries, explicit data scope	Join/denominator/time cases pass; unsupported semantics produce clear clarification or limitations.
3. Daily usability	Compact findings, organized evidence workspace, direct chart edits, freshness/refresh controls, better navigation	Users can verify and refine a result without traversing the full execution history; cosmetic edits need no source execution.
4. Safe expansion	Isolated Python; CSV/Excel as typed artifacts; then explicit approved cross-source bindings	Isolation and lineage checks pass; full populations stay distinct from previews and presentation samples.
5. Repeated value	Saved parameterized analyses, reviewed knowledge updates, scheduled monitoring	Reruns retain scope/version lineage; alerts identify material changes and avoid unchanged-status noise.

File uploads, document research, multi-source analysis, and scheduling are already in the deferred roadmap. They become more useful after verification and interaction improve. For uploads, start with a file as a first-class source before blending it with a warehouse; require explicit join keys and cardinality checks for mixed-source work. External documents should contribute cited context rather than silently redefine governed metrics.

For monitoring, rerun an established analysis specification with fresh evidence and compare against its prior result. Distinguish scheduled deterministic analyses from open-ended recurring investigation. Hex exposes this distinction in its Tasks documentation. [Hex Tasks](#).

The strongest product direction is **an evidence-backed analytical workspace with dependable business semantics and measured analytical quality**. The current implementation already contains much of the execution foundation needed to build it incrementally.

DETAILED REVIEW

Data plugin comparative review

A detailed comparison with the official Data plugin and a practical adoption path.

Data plugin comparative architecture review

Reviewed: 2026-09-14

Scope: the installed Data plugin at `/Users/charlie/.codex/plugins/cache/openai-curated-remote/data-analytics/1.0.8` and the current `data-analytics-agent` repository.

Method: static source and contract review; repository test run; plugin package test attempt. This is an architectural and product-capability review, not an assessment of production data quality or connected-source access.

Executive assessment

The two systems solve complementary parts of an analytics product.

The official **Data plugin** is a broad, host-integrated analytics product layer. Its strongest ideas are explicit task routing, source-authority discipline, reusable guidance, evidence-aware interactive delivery, and rigorous visual/publishing behavior. It deliberately relies on externally connected tools for data access and on a packaged React runtime for reports and dashboards.

This repository is a narrower but more operationally durable analytical execution system. Its strongest ideas are source-bound conversation state, a curated semantic model, strict SQL/Python separation, typed Parquet artifacts, restart-safe runs/checkpoints, idempotent tool commits, human review gates, and deterministic self-contained reporting. It is a better foundation for governed local execution than the plugin's prompt-and-artifact layer.

The recommended direction is **not** to transplant the plugin or replace the current Python/FastAPI/Streamlit architecture. Instead, retain the repository as the execution and evidence plane, then add a small workflow-and-delivery plane inspired by the plugin. Start with intent routing, a richer evidence envelope, and validation; add interactive dashboards only after those contracts are stable.

What was examined

Official Data plugin

- Manifest and README: version `1.0.8`, 20 focused skills, optional connections across warehouse, BI, product analytics, knowledge, messaging, email, calendar, and developer-tool categories.

- Core policy: `skills/index/SKILL.md`, shared source/dependency instructions, analysis-quality and dashboard-quality criteria, and the Data App Contract.
- Delivery system: the protected React/Recharts data-app template, reviewed snapshots, source inspector, editable presentation state, local preview, Sites publishing, selected-view URLs, refresh, export, and a content-addressed build process.
- Quality surface: 705 files; about 132k lines across Markdown, JavaScript/JSX, and JSON; 104 test files in the package.

Repository

- Coordinator, SQL and Python specialists, semantic context, sources/backends, run manager, persistence/stores, presentation, reporting, visualization, API/UI contracts, and test suite.
- Project architecture and operational documentation.

Architecture comparison

DIMENSION	OFFICIAL DATA PLUGIN	CURRENT AGENT	ASSESSMENT
Primary role	Host-integrated analytics workflow and shareable experience	Local, persistent governed execution service	Complementary; keep the agent as the execution system of record.
Task decomposition	20 explicit workflows such as diagnostics, KPI design/readouts, market sizing, data quality, dashboards, reports, notebooks, validation, sharing, and refresh	One coordinator routes to SQL or iterative Python; only analysis and reporting skills are loaded	The agent has a sound compute split but lacks a first-class user-intent taxonomy.
Source access	Connector-agnostic policy across many categories; selects authority by claim and verifies actual access	Configured SQLite/Snowflake sources with one source per conversation and a curated semantic catalog	The agent is safer and simpler for its scope; it needs a capability/authority layer before expanding integrations.
Semantic grounding	Guidance calls for governed definitions, population, grain, period, freshness, and source authority	Curated logical catalog, declared relationships, physical definitions only for SQL, bounded semantic-context tool	A repository strength. Its semantic model is more concrete and enforceable than prompt-only guidance.
Compute model	Delegates to available host/provider tools; no durable local analytic runtime is supplied by the plugin	SQL owns retrieval/description; fresh Python processes own inference/forecasting; Parquet artifacts pass between them	A clear repository advantage for reproducibility, isolation, and nontrivial analysis.
Evidence/provenance	Reviewed source snapshot, query/component IDs, metric definitions, source previews, filter scope, source inspector, reviewed/authoring separation	Saved source/Python/presentation datasets, parent lineage, exact SQL/Python, immutable chart versions, report input references	Both are strong. The agent's lineage is better for execution; the plugin is better for reader-facing provenance and freshness/definition presentation.
Report and dashboard UX	Interactive React apps with editing, source inspection, filters, layout state, export, sharing, publishing, refresh and browser verification	Deterministic self-contained HTML report with Plotly and downloads; Streamlit chat/UI; no first-class dashboard artifact	This is the largest functional gap.
Safety and lifecycle	Explicit source/publishing boundaries, protected runtime, data-secret checks, selected-view links, verification rules	Read-only SQL validation, optional HITL on SQL/Python, strict model filesystem permissions, run budgets, cancellation/resume, checkpoint recovery	The agent is particularly strong for local execution control. The plugin is stronger for delivery and publication guardrails.
Quality model	Written methodology/visual criteria plus an extensive component/browser test corpus	Pydantic contracts, backend/lineage tests, agent-workflow tests, report/chart validation	Add a lightweight structured review gate; do not copy the plugin's very large UI test surface until an interactive UI exists.

Confirmed strengths of the current agent

1. **A clean execution boundary.** `text-to-sql` is the only specialist with warehouse access, while `data-analysis` works from named persisted data artifacts in a fresh Python process. This prevents accidental live-source access from exploratory code and supports reproducible handoffs.
2. **Semantic-model enforcement.** Logical datasets, fields, metrics and declared relationships are bounded before SQL execution. The coordinator sees business-facing metadata while the SQL specialist receives physical definitions only when necessary.
3. **Durable artifacts and resumability.** Results are typed Parquet data, runs are journaled/idempotent, checkpoint state is persisted, interrupted work is recoverable, and reports can be retried without repeating extraction.
4. **Evidence preserves lineage.** `resolve_answer` traverses parent result IDs, Python inputs/outputs, and chart references before publishing. Charts are immutable/versioned and retain display-sampling notes.
5. **Proportionate orchestration.** The coordinator explicitly avoids forcing heavyweight investigation and charts on simple descriptive questions, while allowing multi-step SQL/Python loops for uncertain work.
6. **Defensive report generation.** A strict declarative `ReportSpec`, escaping renderer, contrast checks, local asset embedding, and report-scoped evidence references make offline delivery reliable.
7. **Verified baseline.** The repository test suite completed successfully: 188 passed, 6 skipped in 21.21 seconds. The warnings concern upstream experimental/deprecation notices, not failing behavior.

Confirmed strengths of the Data plugin worth adopting

1. **Intent-first product surface.** It separates product/business analysis, metric diagnosis, KPI design/readouts, market sizing, data quality, notebooks, charts, dashboards, reports, validation, sharing, and refresh. This makes its behavior easier to predict and its guidance more specific.
2. **Authority is evaluated per claim.** The plugin distinguishes primary measurements from supporting business context and treats Slack/messages as leads rather than proof. It requires validation of definition, population, grain, date window, and freshness.
3. **Presentation is evidence-aware.** Its app contract ties every source-backed component to reviewed data/query IDs, preserves filter and presentation state, and offers source inspection rather than merely attaching generic provenance.
4. **Reader-centered reports.** The report workflow is answer-first, uses restrained executive summaries, keeps caveats next to the affected claim, and avoids generic report templates or decorative metrics.
5. **Explicit validation criteria.** Its quality guidance calls out denominator/grain/date checks, independent recomputation, causal overclaiming, all-null versus zero states, accessible visuals, and rendered—not merely build—verification.

6. **Delivery lifecycle completeness.** It has a cohesive model for preview, export to office formats/PDF, sharing, refresh, publication and view-specific links.

Important differences and trade-offs

The plugin is not a drop-in data engine

The plugin package is primarily a workflow specification and frontend/runtime. It expects the host and connected providers to supply source discovery, access, query execution, identity, publishing, and browser control. It should not replace the repository's source registry, SQL safety, Python sandboxing, persistent stores, or semantic catalog.

The agent should not emulate all 20 skills with new agents

The plugin's many workflows are mostly specialized policy and deliverable guidance, not separate compute engines. Creating 20 subagents would duplicate prompts, weaken ownership, and conflict with this repository's deliberate SQL/Python boundary. Use a typed intent router plus focused skill/prompt modules over the existing coordinator and two specialists.

Static HTML versus interactive applications is a real product choice

The agent's self-contained HTML is well suited to auditable local sharing and resilience. The plugin's React surface is better for filtering, layout edits, source inspection, refresh and collaborative refinement. Add an interactive layer only if recurring monitoring and shared exploration are actual user needs; do not trade the current deterministic report for a client-only dashboard without a durable snapshot contract.

Different integration models require different governance

The agent's one-source-per-conversation rule avoids ambiguous joins and authority conflicts. The plugin accommodates many source categories by policy. If the agent expands beyond configured SQL sources, it must introduce explicit authority selection and source-scope records before accepting cross-source evidence.

Findings and practical recommendations

P0 — Fix the plugin package verification experience before treating it as a reliable local build dependency

In this installed cache, `npm test` ran 1,439 tests: **1,407 passed and 32 failed**. Most failures were environmental: missing `react`, `html-to-image`, and `vite` dependencies used by the package's test targets. Two failures also reveal contract drift: a refresh-copy expectation and an uploaded-spreadsheet discovery expectation no longer match the installed source. The plugin documentation claims ordinary authoring builds do not need dependency installation, which is compatible with this finding; its `*test*` command, however, is not hermetic in the installed cache.

Recommendation: do not vendor this cached package as an unpinned runtime dependency. If it becomes a build dependency, consume a release artifact with a verified prebuilt runtime and add a small compatibility smoke test around the exact prepare/build commands the agent will use. Treat its broad test suite as upstream validation rather than a gate for this repository until the dependency closure and test/release contract are fixed upstream.

P1 — Add a typed analytics intent router, not more general-purpose orchestration

Introduce a small `AnalyticsIntent` contract before coordinator dispatch: `lookup`, `descriptive_analysis`, `metric_diagnostic`, `data_quality`, `forecast_or_model`, `kpi_design`, `kpi_readout`, `market_sizing`, `report`, `dashboard`, `notebook`, `validation`, `share`, and `refresh`. Each intent should specify required evidence, permitted sources, delivery type, and whether SQL/Python is expected.

Use it to select concise prompt modules and output validators while retaining the existing SQL and Python specialists. This makes routing inspectable, makes unsupported modes explicit, and allows tests to assert behavior without brittle prompt matching.

Acceptance criteria: a request has one persisted intent, permitted tools are constrained by it, and existing simple totals still follow the one-SQL fast path.

P1 — Enrich the evidence envelope at the artifact boundary

Extend saved-result metadata with structured, optional fields for:

- `captured_at`, `data_as_of`, timezone and freshness semantics;
- metric definition ID/version, formula summary, denominator/grain and policy-effective period;

- source authority tier and a safe source reference/preview;
- applied filters, population/exclusions and requested comparison baseline;
- extraction completeness and material data-quality warnings.

Derive report/chart provenance from this envelope instead of relying on free-form narrative. Preserve the current SQL/Python lineage; this is an additive reader-facing contract, not a replacement.

Acceptance criteria: a chart or metric block can show exactly what data-as-of, definition, population and inputs it represents; missingness is explicit rather than silently absent.

P1 — Add a proportionate validation gate to publication

Borrow the plugin's validation checklist, but make it machine-readable and scoped. Before `publish_findings`, validate only consequential claims:

- definition/grain/period compatibility;
- source completeness/truncation and material null/zero conditions;
- independent calculation where a ratio, contribution, model or surprising result warrants it;
- chart-to-claim agreement; and
- causal wording versus observed association.

Store outcomes as `verified`, `caveat`, `not_applicable`, or `unverified` with a reason. Do not require exhaustive dashboards checks for simple SQL totals. A full deep-review mode can create the plugin-like coverage scorecard only when requested.

Acceptance criteria: publication refuses a central unsupported claim but permits partial findings with named gaps; reports render the material caveats near the relevant claim.

P1 — Make output mode explicit and preserve the existing report path

Add `inline`, `report`, `dashboard`, and `notebook` as output modes. The user-requested form should win; otherwise keep simple answers inline and use a report for multi-finding decision readouts. Current HTML remains the canonical `report` implementation.

Do not make a report mandatory for every scalar result at the domain level. Keep the internal evidence record mandatory; expose the compact report when the product requirement calls for it. This avoids needless artifacts while retaining traceability.

P2 — Build a dashboard as a separate delivery adapter after the evidence contract stabilizes

Add a `DashboardSpec` and `DashboardArtifact`, parallel to—not hidden inside—`ReportSpec`. It should reference immutable results/charts plus the enriched evidence envelope. Start with read-only filters, period controls, source detail, empty/all-null/partial states, and view URLs. Persist presentation separately from reviewed data.

Do not begin by embedding the entire Data plugin template. Use a small adapter layer that can later emit either the current local web surface or a Data-compatible app snapshot. This protects the agent from a large React runtime and keeps its FastAPI/Streamlit boundary modular.

P2 — Introduce an integration capability registry before adding sources

Generalize `DataSource` into a small provider capability contract: `metadata_read`, `metric_definition_read`, `value_query`, `file_read`, `BI_view_read`, `product_event_query`, `message_context_read`, and `publish`. Each provider declares the scopes, freshness semantics, source-link safety, and whether it can support an authoritative measurement versus explanatory context.

Continue to select one controlling source for a quantitative claim. Permit additional sources only when their role is explicit: definition, denominator, explanation, or decision context. This imports the plugin's authority discipline without weakening the current one-source execution rule.

P2 — Add reusable company data context as versioned configuration

The plugin's reusable context capability is valuable: metric definitions, canonical filters, business calendar, reporting conventions, known pitfalls, and approved source links should be shared across conversations. In this agent, implement it as versioned, source-scoped configuration (for example `semantic/context/`) with explicit precedence over free-form user instructions.

Keep it separate from raw observations and checkpoints. Let semantic catalog changes invalidate only the affected new run, as the system already does.

P2 — Strengthen reader-facing report provenance and accessibility

The static renderer should gain compact expandable source/definition panels for each chart/table/metric, a visible `as of` convention, a direct link to safe local evidence downloads, and keyboard-testable disclosure controls. The plugin's source inspector is a good interaction reference.

Keep the self-contained report secure: never embed credentials, signed URLs, unnecessary source rows, or arbitrary remote scripts. The agent already has a good foundation here through escaped deterministic HTML and local assets.

P3 — Add lifecycle adapters only when users need them

After dashboards exist, add explicit adapters for PDF, DOCX, slides, sharing, and recurring refresh. Refresh must reopen the same artifact, rerun saved source requests through the existing source backend, validate changed claims, preserve user presentation state, and never silently widen access.

Avoid coupling this to local desktop availability. A scheduled refresh needs a durable execution environment and credentials that can be re-authorized; it cannot be a background thread in the current one-process local deployment.

Recommended target shape

MERMAID

```
flowchart LR
    I[User request] --> R[Intent router]
    R --> A[Authority and capability policy]
    A --> C[Existing coordinator]
    C --> S[Existing text-to-SQL specialist]
    C --> P[Existing Python analysis specialist]
    S --> E[Immutable evidence envelope]
    P --> E
    E --> V[Scoped validation gate]
    V --> F[Published findings]
    F --> H[Existing deterministic HTML report]
    F --> D[Future dashboard adapter]
    E --> X[Source/detail inspector]
```

The important boundary is between **execution/evidence** and **delivery**. The current system should remain authoritative for SQL, Python, semantic definitions, artifacts, approvals, and run state. The new layer should select workflows, represent reader-facing provenance, validate claims, and package the same immutable evidence.

Delivery plan

PH ASE	SCOPE	WHY NOW	EXIT CRITERIA
1	Intent router, output modes, evidence-envelope schema	Improves predictability without changing compute or UI	Existing tests pass; new route tests cover all intents and SQL/Python ownership remains unchanged.
2	Scoped validation and report source/definition disclosures	Raises analytical trust with low product risk	Consequential reports record validation outcomes and display material caveats; partial reports remain usable.
3	Provider-capability registry and versioned Data Context	Enables controlled integration growth	A second contextual source can be labeled as explanatory without being mistaken for a metric authority.
4	Read-only dashboard artifact	Solves monitoring/exploration demand	Saved view state, filter semantics, source details and empty states are testable from immutable snapshots.
5	Export, sharing, hosted refresh	Adds operational distribution	Refresh reproduces a stored artifact scope and preserves provenance and presentation state.

Review limits

- No live warehouse, BI, product analytics, document, or messaging connector was available or needed for this code/package review.
- The plugin's browser/visual tests were not run because the installed cache lacks the required local test dependencies; no claim is made about its browser runtime behavior.
- This repository is intentionally single-user and local. Public URLs, multi-user authorization, and hosted scheduling should be designed as new product boundaries, not inferred from the current API.

Bottom line

Adopt the Data plugin's **product contracts**—intent routing, source authority, evidence-rich presentation, scoped validation, and artifact lifecycle—while preserving the agent's **execution contracts**—semantic enforcement, SQL/Python separation, typed artifacts, persistence, budgets, approvals, and deterministic rendering. That combination will produce a substantially more complete analytics product without sacrificing the reliability already present in this repository.

Generated from the two canonical review files in this repository. This packet has no external fonts, scripts, analytics, or network dependency.